CORAL-DAOD6000 Documentation (draft)

Overview / Introduction

CORAL is a domain-specific GraphRAG (Graph Retrieval-Augmented Generation) prototype designed to support question answering over Canadian Department of National Defence (DND) policy documents, with a focus on the DAOD 6000-series (Information Management / Governance related directives).

The project combines:
- Document scraping and parsing
- Structured chunking (Document → Section → Chunk)
- Knowledge graph modeling in Neo4j
- Embeddings + vector search
- Keyword/full-text retrieval
- Hybrid retrieval strategies
- LLM-based answer generation

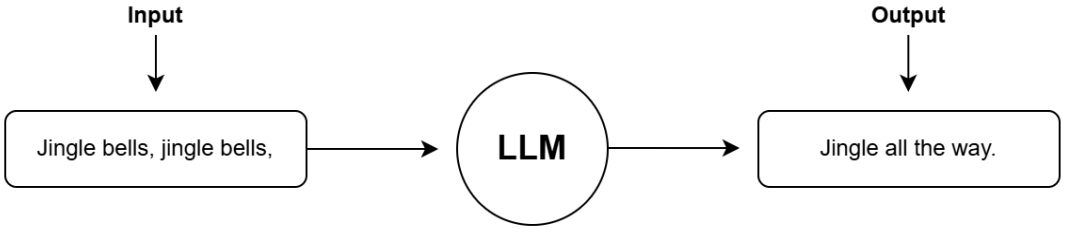
The goal is to improve policy lookup and understanding by enabling natural-language questions such as:

- “What is ADM(Fin)/CFO responsible for regarding electronic authorizations?”
- “Who approves the implementation and use of SES?”
- “What controls are required for financial transactions?”

Instead of relying on simple keyword search alone, CORAL uses a graph-aware pipeline and multiple retrieval strategies to return relevant context before generating a response.

What are LLMs (Large Language Models)?

Large Language Models (LLMs) are AI models (like the ones behind ChatGPT) designed to understand and generate human-like text. They are typically built using the transformer architecture, and they learn by training on very large collections of text. During training, an LLM mainly learns to predict the next word/token in a sequence, which allows it to pick up grammar, patterns, context, and some degree of reasoning-like behavior.



Even though LLMs can produce highly convincing answers and follow detailed instructions (for example, generating a haiku in a specific style), they do **not** store facts as a normal database would. Instead, they produce responses using **learned statistical patterns (model weights)** from training data, which can lead to important limitations:

- **Knowledge cutoff / outdated info:** they may not know recent events or updates.
- **Hallucinations:** they can confidently generate incorrect or fabricated details.
- **No private knowledge by default:** they can't answer questions about internal/proprietary documents unless that information is provided to them.

These limitations motivate approaches like **Retrieval-Augmented Generation (RAG)**, where the model is given relevant external information at question time so it can respond using grounded, up-to-date, or private context.

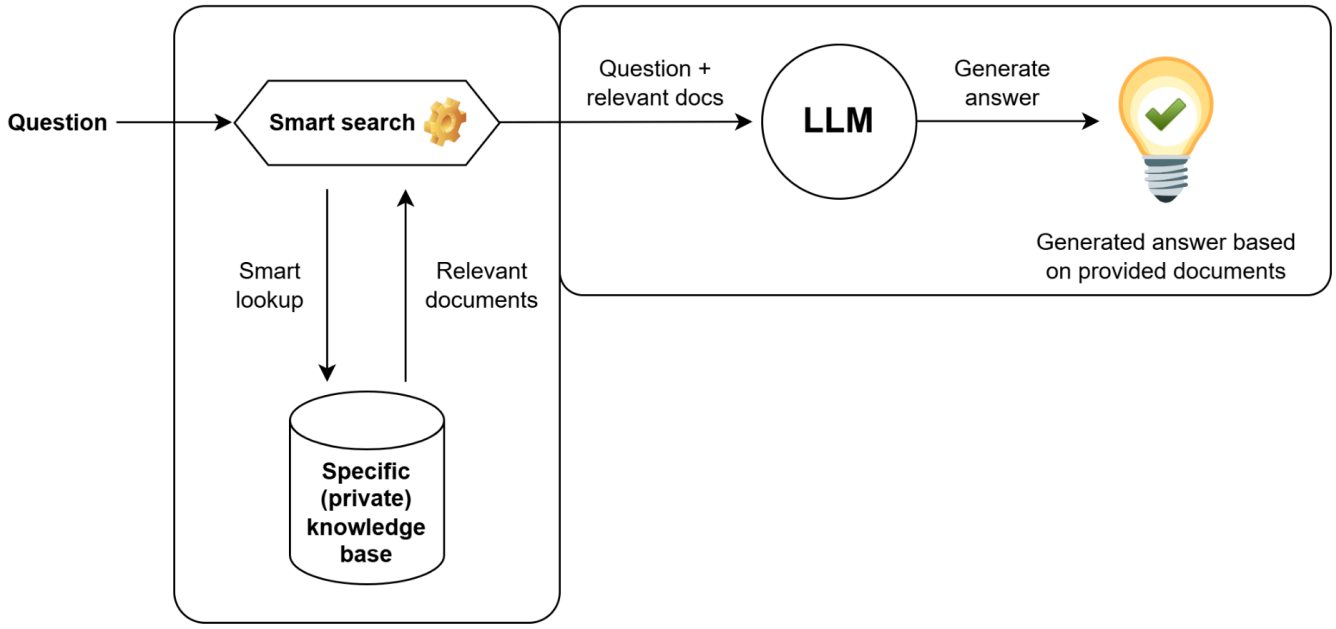
What is RAG (Retrieval-Augmented Generation)?

Retrieval-Augmented Generation (RAG) is a workflow where a language model answers a question using external, retrieved information instead of relying only on what it learned during training.

How it works:

1. Retrieval: the system searches an external knowledge base (docs, PDFs, database, etc.) to find the most relevant passages.
2. Augmented generation: those passages are inserted into the model's prompt/context, and the model generates an answer grounded in that provided context.

Why it's used: improves factual accuracy, reduces hallucinations, and allows responses to reflect updated or private information.



What is a Graph Database?

A graph database stores data as a graph, meaning:

- **Nodes** represent things (e.g., *Document, Section, Chunk, Entity, Person, Organization*).
- **Relationships (edges)** connect nodes and describe how they're related (e.g., *HAS_SECTION, CONTAINS, MENTIONS*).
- Both nodes and relationships can have **properties** (key–value fields like title, id, date, text, etc.).

Unlike relational databases (tables + joins), graph databases are built for questions that depend on connections, such as:

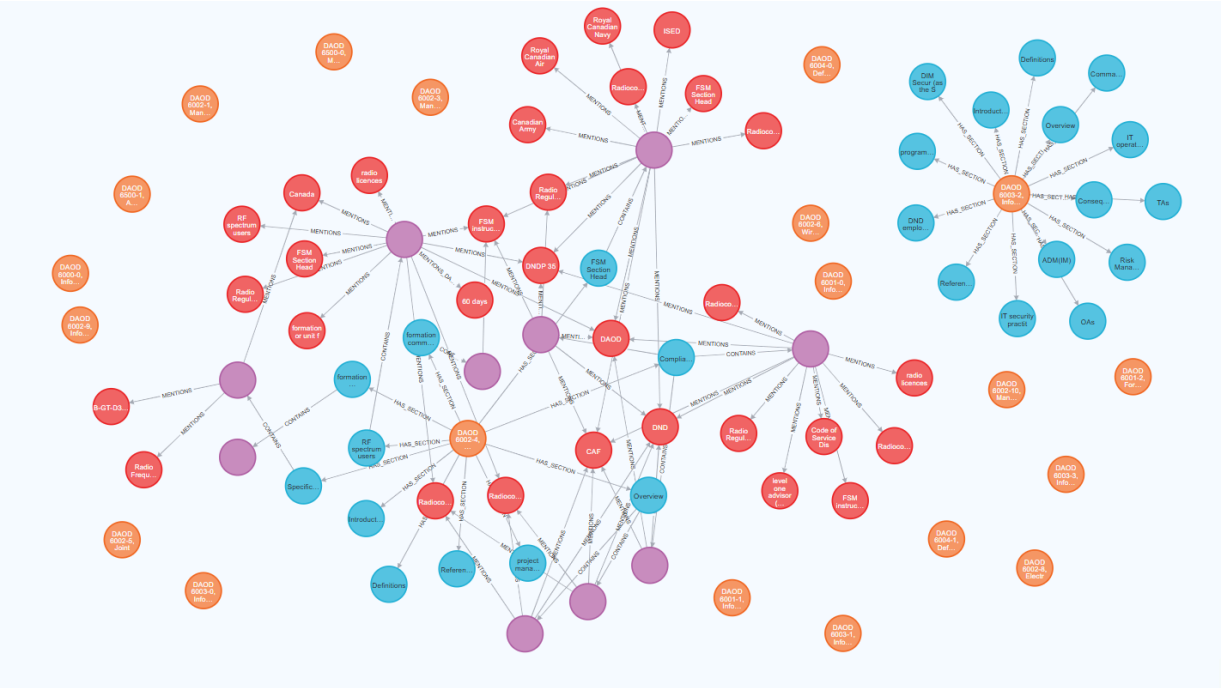
- "Which sections mention this entity?"
- "What chunks belong to this document section?"
- "Which responsibilities are linked to this role across policies?"

Because relationships are first-class and easy to traverse, graph databases are especially useful for highly connected data and for building systems like GraphRAG, where retrieval benefits from following links between related concepts and source text.

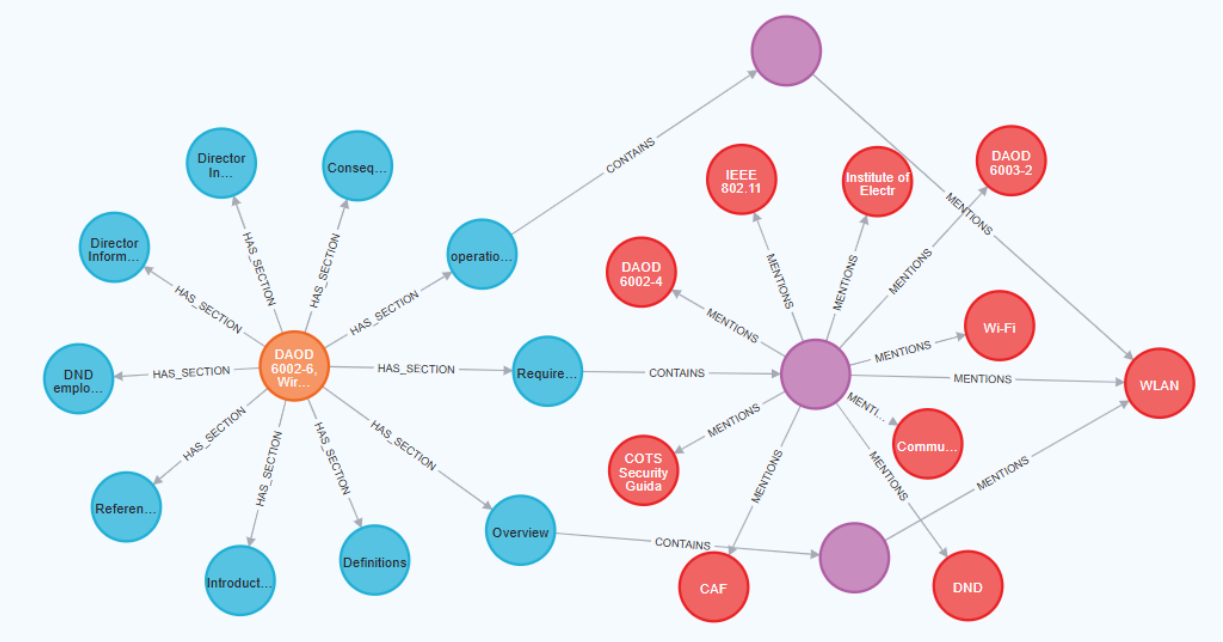
What is GraphRAG (Graph Retrieval-Augmented Generation)?

GraphRAG is a type of RAG where the external knowledge base is organized as a knowledge graph (nodes + relationships). Instead of retrieving only the most similar text chunks, GraphRAG uses the graph structure to retrieve connected, structured context that better matches how information is linked in the real world.

Knowledge graph created with Neo4j:



Close up of a partially expanded node (where the orange node represents the root node):



How the graph works (Neo4j)

Node types
CORAL stores policy content and extracted knowledge as nodes:

- **Document**: one DAOD document (DAOD number, title, URL, metadata)
- **Section**: a section inside a document (heading + text)
- **Chunk**: a small passage of section text used for retrieval + embeddings
- **Entity**: extracted “things” mentioned in text (roles, orgs, systems, etc.)

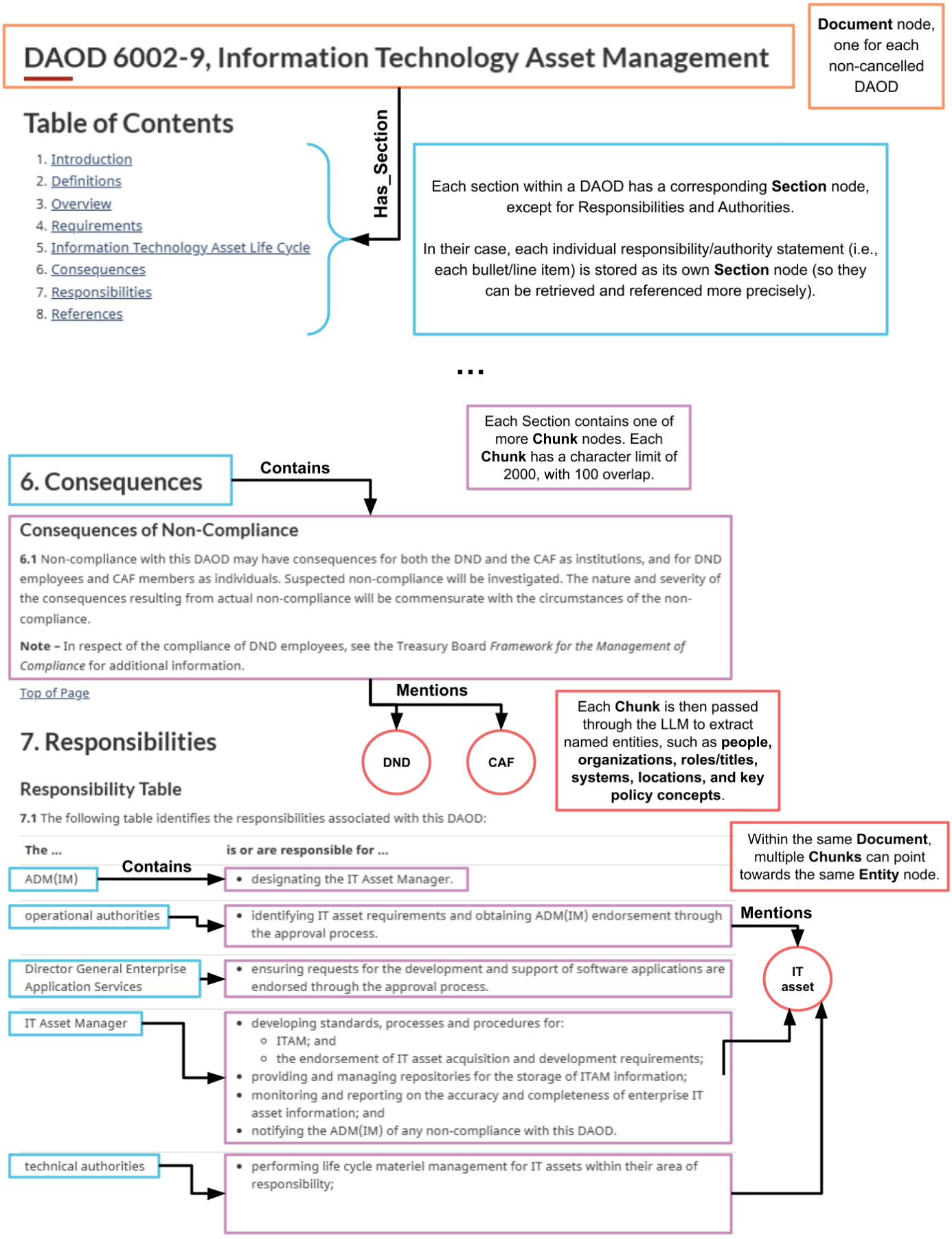
Relationships
CORAL keeps the original hierarchy and links extracted knowledge:

- (:Document)-[:HAS_SECTION]->(:Section)
- (:Section)-[:CONTAINS]->(:Chunk)
- (:Chunk)-[:MENTIONS]->(:Entity)

Properties

- Document (:Document:: id, title, author, text, language, subjects, effective_date, published_date, retrieved_at, url, canonical_url, domain, hash
- **Section** (:Section): id, section_index, title
- **Chunk** (:Chunk): id, section, text, start_char, end_char, token_count, significance, embedding, embedding_model, embedded_at, extract_model, extracted_at
- **Entity** (:Entity): id, name, type, aliases, source_doc_id.

How documents are being stored in the db



Data Retrieval & Generation

CORAL supports multiple retrieval strategies to gather relevant context before the LLM generates an answer.

- **Keyword Retriever** (retrieve_keyword)
Finds relevant chunks using keyword / full-text matching. If it detects a DAOD in the user prompt, it will only return chunks from the corresponding DAOD.

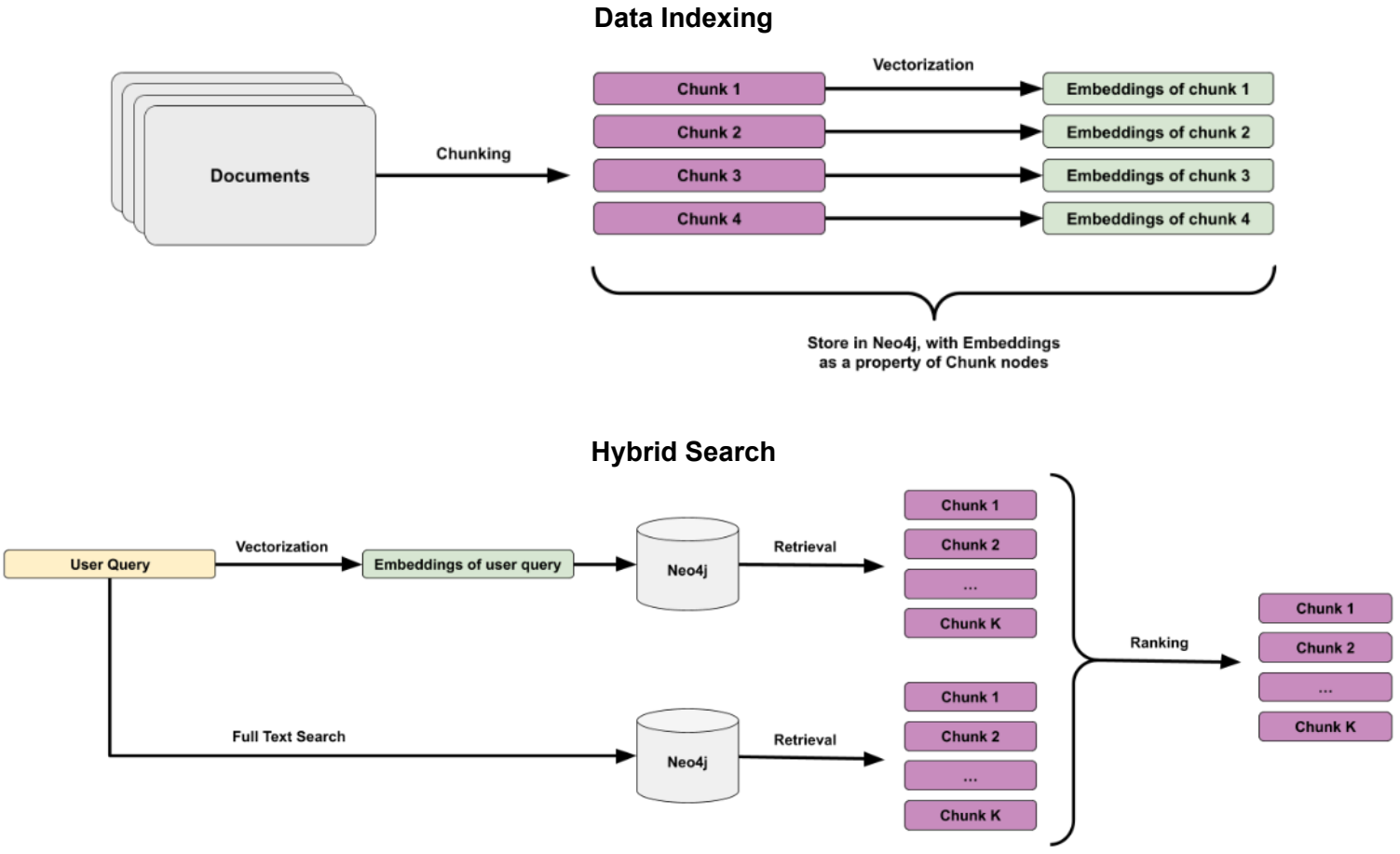
- **Entity Retriever** (retrieve_entity)
Uses detected entities (people/orgs/concepts) and traverses the graph to pull related chunks.

- **Hybrid Retriever** (retrieve_hybrid)
Combines keyword signals with semantic/vector similarity (embeddings) to improve recall and relevance.

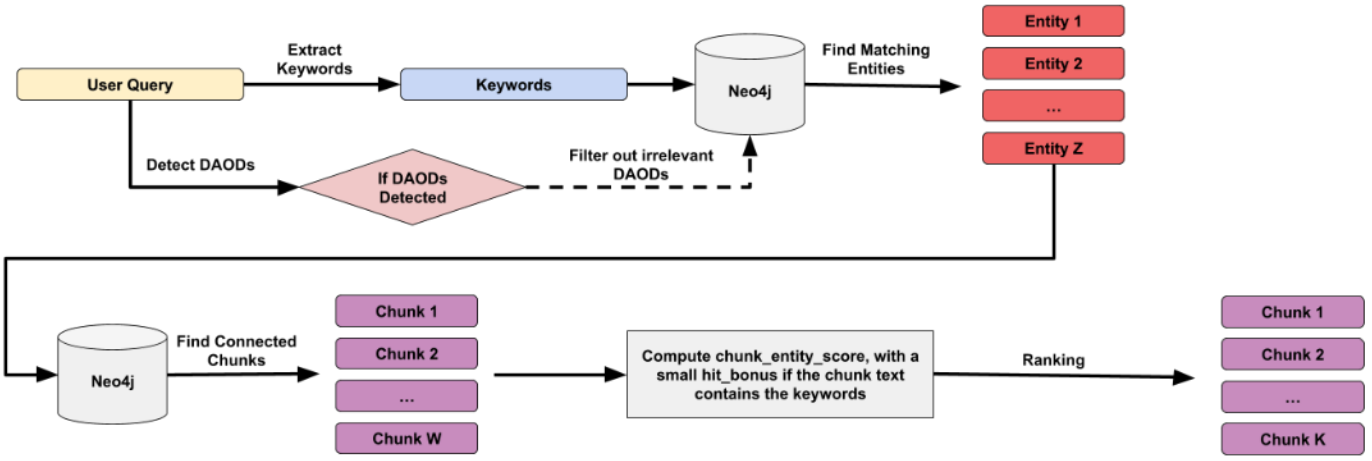
- **Keyword-hybrid Retriever** (retrieve_keyword_hybrid)
Uses Hybrid search and Keyword search, then combines the results (may need some fine tuning)

After retrieval, the selected passages are inserted into the prompt, and the LLM (via Ollama) generates a grounded response based on that retrieved context.

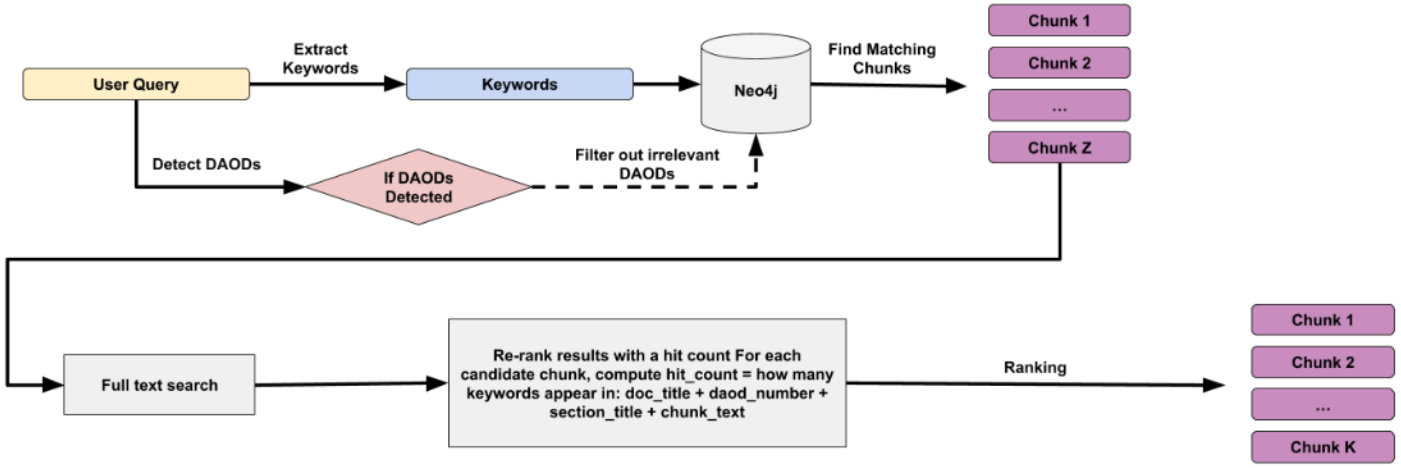
Basic RAG Pipeline:



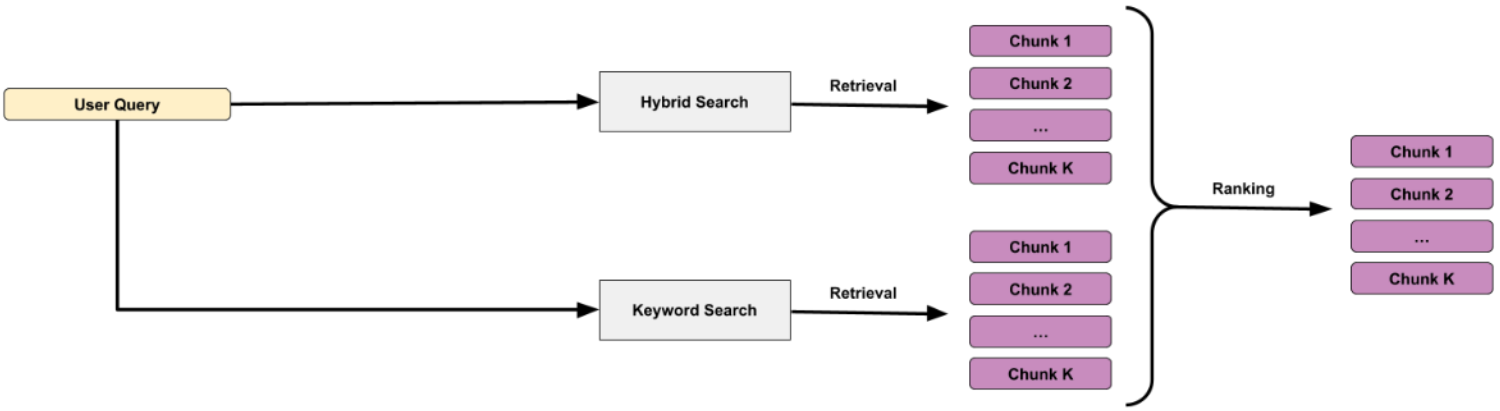
Entity Search



Keyword Search



Keyword-Hybrid Search



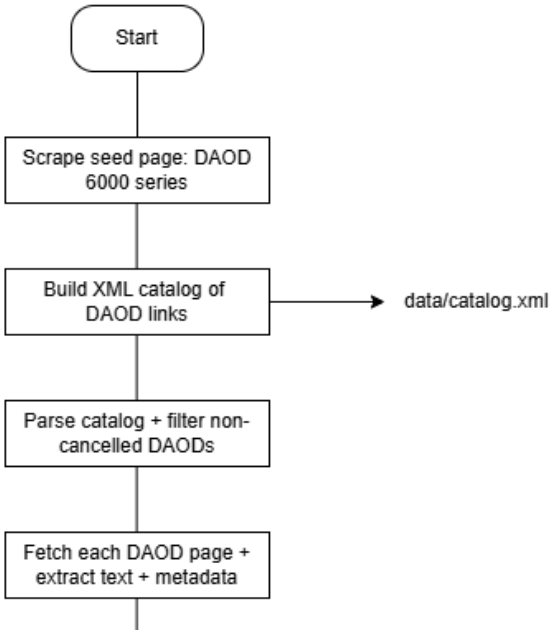
Architecture & Design

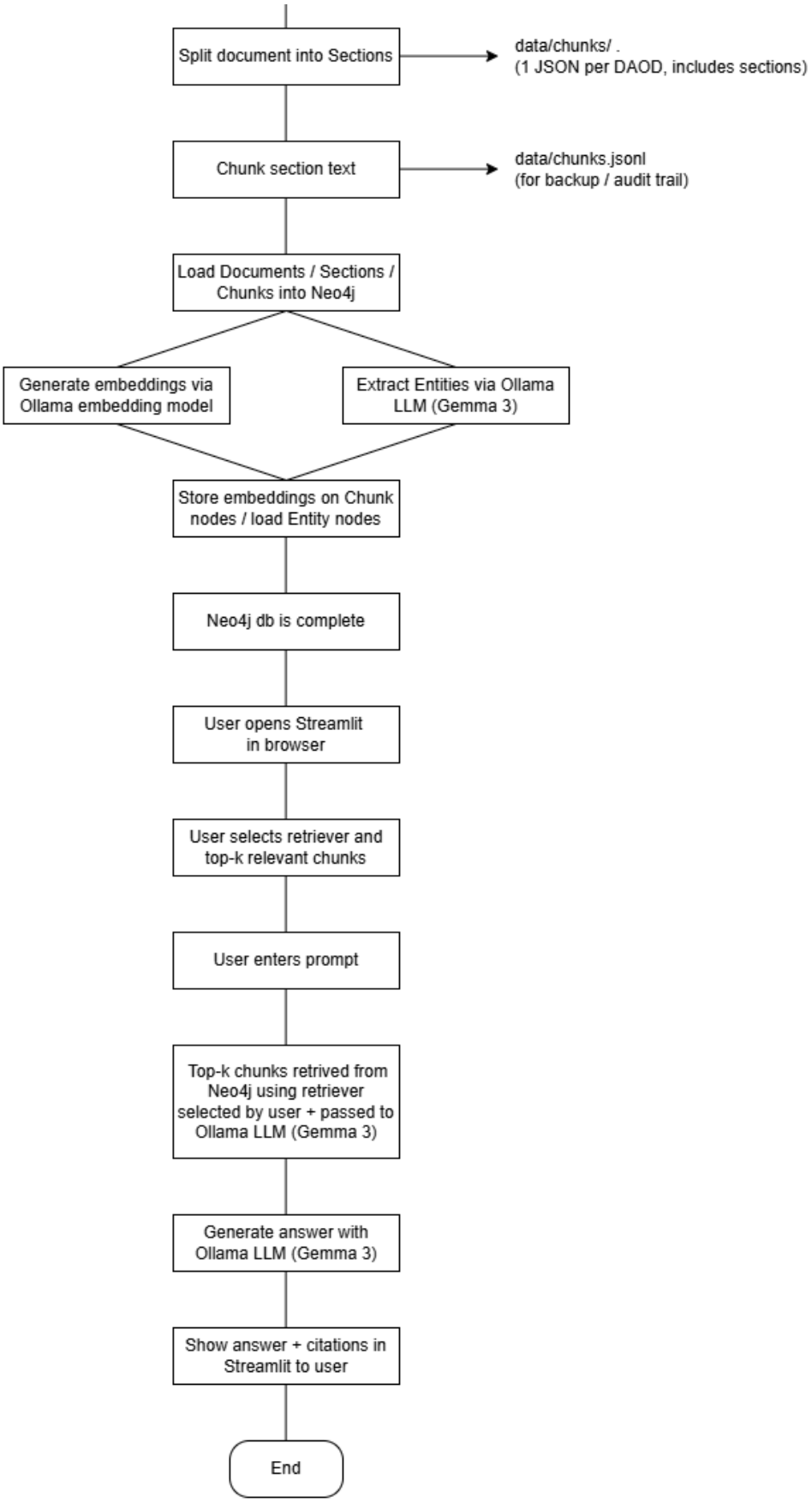
1) One-time database build (ingestion pipeline)
This is the “setup” phase. You run it when Neo4j is empty or when you want to rebuild the database.

1. Scrape DAOD links and write them to an XML catalog → `catalog.xml`
2. Parse the catalog, keep only **non-cancelled** DAODs, fetch the pages, and split into Sections
   - Output: per-document JSON files in `docs_json/`
3. Chunk the section text into smaller passages
   - Output: `chunks.jsonl` (backup)
4. Load Document/Section/Chunk into Neo4j
5. Generate & store embeddings for chunks using Ollama embeddings
6. Generate & load entities using Ollama LLM

2) Asking questions (runtime Q&A flow)
This is what happens every time a user asks a question in Streamlit (or via CLI).

1. User enters a **question**
2. User selects a **retrieval mode** (keyword / entity / hybrid / keyword-hybrid)
3. The retriever queries Neo4j to find the **top-K most relevant chunks**
4. (Optional) expand chunk hits to **full section context**
5. Build the LLM input:
   - assemble the retrieved context
   - attach **sources/citations** (DAOD + section + chunk id, etc.)
6. Call **Ollama LLM** to generate the final answer
7. Display:
   - the answer
   - the sources/citations used




Split document into Sections
data/chunks/ .
(1 JSON per DAOD, includes sections)
Chunk section text
data/chunks.jsonl
(for backup / audit trail)
Load Documents / Sections / Chunks into Neo4j
Generate embeddings via Ollama embedding model
Extract Entities via Ollama LLM (Gemma 3)
Store embeddings on Chunk nodes / load Entity nodes
Neo4j db is complete
User opens Streamlit in browser
User selects retriever and top-k relevant chunks
User enters prompt
Top-k chunks retrived from Neo4j using retriever selected by user + passed to Ollama LLM (Gemma 3)
Generate answer with Ollama LLM (Gemma 3)
Show answer + citations in Streamlit to user
End

Configuration & Deployment Guide

Please refer to readme.md

Technology Used

Tech	**Description**
Git, Github	Versioning, code sharing
VSCode	Dev IDE
Neo4j	GraphDB
Cypher	Neo4j query language
Ollama LLM	Local LLM + embeddings
Streamlit	Light weight web app UI framework
Docker	Containerization +Deployment
Harbor	Vulnerability testings

Evaluation: to be completed

Lessons Learned: to be completed